

```

from pathlib import Path
import pandas as pd
from imblearn.over_sampling import SMOTE
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
# Find the dataset from the current folder or one of its parent folders
possible_paths = [Path.cwd() / "creditcard.csv"]
possible_paths += [parent / "Practical 10" / "creditcard.csv" for parent in Path.cwd().parents]
data_path = next((path for path in possible_paths if path.exists()), None)
if data_path is None:
    raise FileNotFoundError("creditcard.csv was not found. Place it in the Practical 10 folder.")
data = pd.read_csv(data_path)
X = data.drop(columns="Class")
y = data["Class"].astype(int)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)
# Scale features using only the training data
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
# Apply SMOTE only to the training data to avoid test-set leakage
smote = SMOTE(random_state=42)
X_train, y_train = smote.fit_resample(X_train, y_train)
print(f"Dataset shape: {data.shape}")
print(f"Fraud transactions before SMOTE: {y.sum()} ({y.mean() * 100:.2f}%)")
print(f"Training class counts after SMOTE: {y_train.value_counts().to_dict()}")

```

```

Dataset shape: (284807, 31)
Fraud transactions before SMOTE: 492 (0.17%)
Training class counts after SMOTE: {0: 227451, 1: 227451}

```

```

import tensorflow as tf
from sklearn.metrics import classification_report, confusion_matrix
# Build a simple deep neural network
model = tf.keras.Sequential([
    tf.keras.layers.Input(shape=(X_train.shape[1],)),
    tf.keras.layers.Dense(32, activation="relu"),
    tf.keras.layers.Dropout(0.2),
    tf.keras.layers.Dense(16, activation="relu"),
    tf.keras.layers.Dense(1, activation="sigmoid")])
model.compile(
    optimizer="adam",
    loss="binary_crossentropy",
    metrics=["accuracy"])
history = model.fit(
    X_train,
    y_train,
    validation_split=0.2,
    epochs=10,
    batch_size=2048,
    verbose=1)
# Convert probabilities to class predictions
probabilities = model.predict(X_test, verbose=0).ravel()
predictions = (probabilities >= 0.5).astype(int)
print("\nClassification report:")
print(classification_report(y_test, predictions, target_names=["Genuine", "Fraud"], zero_division=0))
print("Confusion matrix:")
print(confusion_matrix(y_test, predictions))

```

```

Epoch 1/10
178/178 ————— 2s 4ms/step - accuracy: 0.9162 - loss: 0.2379 - val_accuracy: 0.9031 - val_loss: 0.1823
Epoch 2/10
178/178 ————— 1s 3ms/step - accuracy: 0.9600 - loss: 0.1001 - val_accuracy: 0.9311 - val_loss: 0.1234
Epoch 3/10
178/178 ————— 1s 3ms/step - accuracy: 0.9703 - loss: 0.0732 - val_accuracy: 0.9601 - val_loss: 0.0845
Epoch 4/10
178/178 ————— 1s 3ms/step - accuracy: 0.9769 - loss: 0.0585 - val_accuracy: 0.9736 - val_loss: 0.0640
Epoch 5/10
178/178 ————— 1s 3ms/step - accuracy: 0.9811 - loss: 0.0485 - val_accuracy: 0.9839 - val_loss: 0.0487
Epoch 6/10
178/178 ————— 1s 3ms/step - accuracy: 0.9843 - loss: 0.0412 - val_accuracy: 0.9891 - val_loss: 0.0375
Epoch 7/10
178/178 ————— 1s 3ms/step - accuracy: 0.9868 - loss: 0.0355 - val_accuracy: 0.9912 - val_loss: 0.0326
Epoch 8/10
178/178 ————— 1s 3ms/step - accuracy: 0.9888 - loss: 0.0314 - val_accuracy: 0.9950 - val_loss: 0.0252
Epoch 9/10
178/178 ————— 0s 3ms/step - accuracy: 0.9907 - loss: 0.0274 - val_accuracy: 0.9966 - val_loss: 0.0211
Epoch 10/10
178/178 ————— 0s 3ms/step - accuracy: 0.9920 - loss: 0.0246 - val_accuracy: 0.9988 - val_loss: 0.0181

```

```
Classification report:
      precision    recall  f1-score   support

   Genuine         1.00      1.00      1.00    56864
    Fraud         0.24      0.86      0.37      98

 accuracy              0.99    56962
macro avg              0.62      0.93      0.68    56962
weighted avg          1.00      0.99      1.00    56962
```

```
Confusion matrix:
[[56593  271]
 [  14   84]]
```